

Generative AI For Synthetic Positive Case Creation

Clayton Becker¹, Anirudh Divecha¹, Hope (Hung) Lin¹ and Daniel Vasilevko¹

¹University of California, Los Angeles

Abstract -

We predict advertisement click-through using demographic attributes, user news-site behavior, and historical ad interactions from a highly imbalanced Kaggle dataset in which only 1.5% of records reflect a click. Standard models, including logistic regression, regularization, LightGBM, and a neural network, perform poorly, with best F1 scores around 0.23. We hypothesize that extreme class imbalance limits signal about the positive class and explore synthetic data generation using CTGAN, DataSynthesizer, and LLM-based methods. Despite generating large volumes of synthetic minority-class examples and testing multiple integration strategies, synthetic augmentation did not markedly improve performance, and models trained on synthetic data only degraded substantially.

1 Introduction

From a business standpoint, it is important for marketing to be targeted in order to be effective. For instance, knowing the ad placement, design, and content that are most associated with a user’s interest would allow businesses to develop stronger marketing strategies that are more cost-effective. In this project, we attempt to predict individuals’ advertisement click-through rates based on demographic data, information taken from their activity in news sites, along with known click rates on past ads.

1.1 Dataset

The original data, found on Kaggle, consists of training and testing data, each containing information from two domains: the source domain (news feeds) and the target domain (advertisement behavior) (7). The news feeds have information about users’ phones, geographical location, browser usage, and click data on news articles, among other things, having a total of 28 features. The advertisement data contains information on users’ behavior with specific ads, with detailed fields including but not limited to ad design, content, and location type on the site, along with timestamps and user device information, containing a total of 35 features. Only 1.5% of the data records a “click” on an ad, resulting in severely unbalanced data.

1.2 Initial Approach

We began with a typical modeling approach, running logistic regression, regularization, light gradient boosting, and neural networks. These methods all performed relatively poorly. The most robust model, the LightGBM, only achieved an F1-Score of 0.23. We hypothesize that a primary reason for this low predictive power is the unbalanced nature of the data.

1.3 Updated Approach

In order to combat this class imbalance we test various methods of synthetic data generation with the intent of making up for the class imbalance in order to train a more robust model.

2 Pre-processing Feature Selection

The ads data and the feeds data were not initially identified at the same level. We therefore flatten the news feeds data so that each row is uniquely identified by a $user_id$ rather than a combination of a $user_id$ and article. We retain click behavior and article progress for articles as separate columns. We keep only the first five articles as the number of users with more than five drops off rapidly making anything past 5 only serve to increase the computational expense of running the data. Additionally, we handle highly cardinal data by collapsing values with fewer than 1000 observations in the aggregated data. We then merge rows using $user_id$ as the join key, making the assumption that the users appearing in both sets are indeed the same users, and split the data along an 80-20 split into training and testing as the original testing set is actually missing the label.

3 Modeling without Synthetic Data

For all models run without synthetic data generation we perform a 80/20 train-test split, and use the F1 score instead of accuracy as our metric for performance because of the class imbalance.

3.1 Baseline Model

Logistic regression is a method that turns a binary classification problem into one that can be modeled by a linear relationship between predictors. Specifically, it defines the log-odds as a function of a linear transformation of predictors, thus minimizing the log loss (8). We use a logistic regression model (without regularization) as our baseline using the full set of predictors, using 0.1 as a threshold for predictions of the positive class to make up for the extreme class imbalance (for reference, using 0.5 as a threshold results in only 4 positive labels). With this, training only on the ads dataset, we get an F1 score of 0.190 on the training data, and 0.133 on the testing data.

When we also include data from the news feeds, training F1 is 0.102, and testing F1 is 0.063. We determined that overfitting likely occurred, suggested by the higher training F1 as compared to testing F1.

3.2 Regularization

Regularization methods (or shrinkage methods) are commonly implemented to handle multicollinear data and improve model interpretability (9). It reduces the variance in the model, at the cost of introducing some bias. By introducing a penalization term to the typical regression problem, we optimize over the squared error with a penalty on the size of beta coefficients. In ridge regression, the penalty is the beta coefficients squared, while in lasso regression, the penalty is simply the absolute value of the coefficients. Such a model implies that ridge regression shrinks coefficients toward zero (making them smaller without being exactly zero), while lasso brings them to zero. Note that all the predictors are standardized in order for the penalty term to be applied without units being an influence.

Because we overfit in our logistic model, have so many features, and are interested in an interpretable model from a business standpoint, we perform lasso regularization. We were able to decrease the amount of significant predictors from 1488 to 244. However, the F1 score on testing data was a paltry 0.022.

3.3 Neural Network

We test a neural network (NN) in order to see if the neural network could adapt well to this imbalanced dataset and improve score performance. Pre-processing for the neural network was slightly different and it included scaling numeric features to zero-mean and unit variance and one hot encoding categorical features.

We developed a very simple three layer neural network using TensorFlow/Keras in Python in order to test performance. The network architecture details are provided below:

Neural Network Architecture in TensorFlow language:

Input Layer (N features) → Dense (128 neurons, ReLU activation) → Dropout (0.3) → Dense (64 neurons, ReLU activation) → Dropout (0.2) → Output (1 neuron, sigmoid activation)

When creating a neural network architecture, we needed to define an activation function for each of the layers. For the first two hidden layers we used ReLU for nonlinearity since it is the most commonly used activation function. For the output layer we used sigmoid since we are working on a binary classification problem. Dropout is another regularization technique that is applied to reduce overfitting. Dropout works by randomly deactivating a subset of neurons during training in

order to get the network to generalize better. Lastly, we applied the ADAM optimizer since it is the widely used optimization algorithm for training neural networks.

After designing our architecture, we proceeded with training in which we set up to have 30 epochs, batch size of 512, and parameterizing our NN to consider the output class as imbalanced.

After training and testing our NN, we found the accuracy to be 98% but this was also not indicative of the model's performance as when we analyzed the confusion matrix we found that our neural net predicted all 0s. This is shown in Figure 1 below.

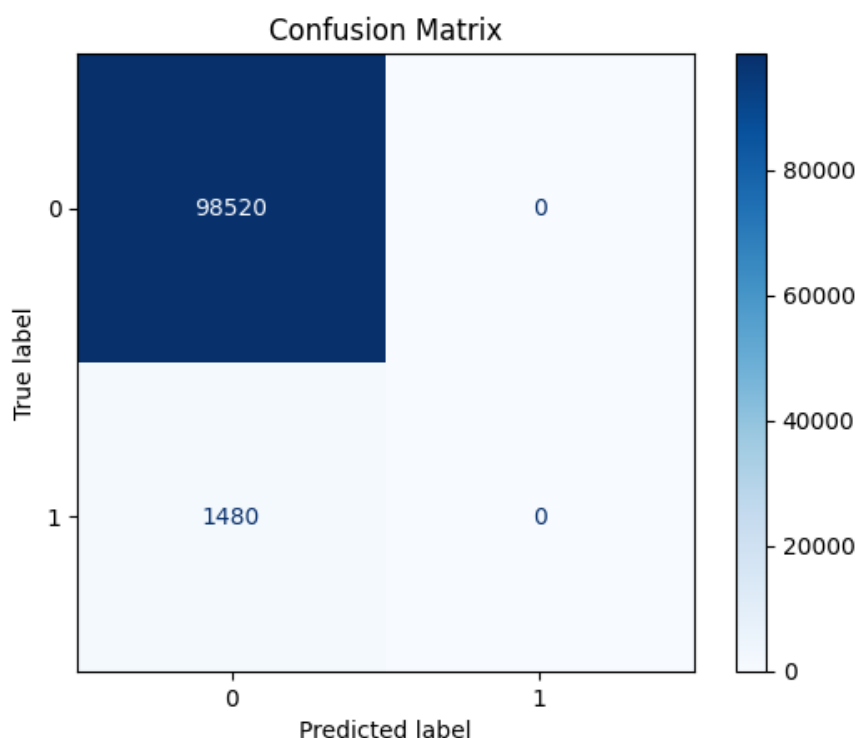


Figure 1. The neural network analysis predicted solely the majority class.

3.4 Light Gradient Boosting Machine

The most successful technique we employed was the Light Gradient Boosting Machine. The Light GBM combine gradient descent with a boosting modeling approach (1). Light GBM is a variation of the decision tree model. Initially, it converts all data into “bins”, similar to a histogram, to reduce the computational expense of sifting through high cardinality data. Once it has done this with all predictors it finds the predictor that has a split that minimizes error and creates branches A and B. Then, rather than splitting each branch again it only splits the branch that would reduce error the most. Let's say B into C and D. For the 3rd split it looks at A, C, and D and only splits the most impactful branch. This method is called a “leaf-wise” strategy and is the reason this method

is called “light”. By reducing the number of splits, the model uses less computation power than other methods such as XGBoost or Random forest making it ideal for our large data set. This model resulted in an F1 score of 0.232.

As a result, we carry this modeling approach forward to assess the success of the generative processes in helping us rectify the problems posed by the class imbalance.

4 Synthetic Data Generation

Synthetic generation of tabular data is a highly involved process, as methods must learn nuanced relationships between heterogeneous features and create tabular datasets that are representative of these complex, high-dimensional distributions (5). This is especially tricky when considering data that contain both numerical and categorical features, as ours does. Additionally, generated datasets are not immediately ready for use; a common problem is that created datasets contain non-commonsense values, and so manual post-processing, with a strong understanding of each feature is often required to further refine datasets for model use.

Synthetic data generation in the context of modeling has been shown to improve model utility for downstream tasks, as compared to models trained only on the original data (2). Additionally, in this case of severely imbalanced classes, a standard way to address this is by oversampling the minority class with synthetic data (3). In the second half of this project, we investigate three models of synthetic generation, CTGAN, DataSynthesizer, and an LLM-based approach, to create artificial positive observations which we then use to augment the human generated data.

Beyond the promise of creating more observations of the positive class, another motivation for synthetic data generation is to maintain the privacy of the original data, such that from the newly generated data, it is impossible to identify features of the original observations (10). This is done by injecting degrees of noise to estimated distributions (4). Here, we implement differential privacy in our use of the DataSynthesizer.

4.1 CTGAN

Conditional Tabular Generative Adversarial Network (CTGAN) is a type of General Adversarial Network (GAN) that is conditioned on discrete columns during training (6). GANs are a model of data generation in which a generator is trained to create data that is able to fool a discriminator, which tries to distinguish between real and synthetic data (11). Thus the problem is reformulated as an optimization problem that involves both a generator and discriminator (5).

CTGAN, developed by Xu et al. (2019) [13], and has substantial advantages over traditional GAN models, including but not limited to being able to work with heterogeneous data, handle unbalanced data substantially better, and also implementing differential privacy, though in this

case we choose not to do so, relying instead on the purpose built DataSynthesizer, which is more appropriately tailored for differential privacy. (6).

Due to the computational demands of training the CTGAN model on such a large dataset with such a high number of features, we train using 30 epochs and a batch size of 1000. This is a smaller number of epochs than is considered traditional, but we believe still appropriate given the higher amount of data seen in each epoch.

Once trained, we make use of the CTGAN in two ways. First, we generate a synthetic dataset of 500,000 rows which maintains the original datasets' distribution of the positive and negative classes. Second, we generate a synthetic dataset of 200,000 positive observations in an attempt to balance the categories when mixed with the real data. This second dataset was constructed by continuously generating batches of 10,000 synthetic observations and only retaining the positive cases until we reach 200,000 rows. This avoids having to sample only the same positive cases over and over and should, in theory, provide a more realistic set of synthetic 1's.

4.2 DataSynthesizer

Machine learning methods of synthetic tabular data generation consist of Gaussian Mixture Models and Bayesian networks (5). DataSynthesizer, developed by Ping et al. (2017) [14] is one of the best known Bayesian methods of data generation, in which the data distribution is explicitly parametrized by a Bayesian network (4). There are challenges with using these Bayesian approaches in cases such as ours. One of the common challenges is the difficulty in accurately estimating the complex, multi-dimensional probability distributions present, thus limiting its potential for synthetic data generation of real world data (5). An additional limitation of the DataSynthesizer for our purposes is that it can only handle numeric data, and does not work at all with categorical data types. As a result, we have to drop certain columns that the CTGAN handles easily from the DS process.

However, despite these limitations, DS is one of the most classically employed statistical methods in the field, and as such, is extremely conducive to model interpretability. As a result, we implement it to provide a comparison to the CTGAN despite our prior that it will likely be less accurate.

The python package DataSynthesizer is able to generate artificial data from a given dataset through a three-step process: first, by describing the data in detail and adding noise, then by sampling from the distribution to generate data, and finally by inspecting the quality of the generated data, allowing for downstream changes (4). For our purposes, we use the package to learn the parametric distribution of the data and then generate samples, with and without differential privacy.

For each privacy level, we generate 500,000 rows with the original distribution of 1s and 0s,

and 50,000 rows with only 1s. We limited ourselves to 50,000 rows rather than the 200,000 of the CTGAN due to the time that it takes to generate positive cases. Due to the class imbalance of the original dataset, the Bayesian process produces very few positive observations in each batch, meaning that generating 200,000 rows would take several hours. As it is, generating 50,000 positive cases took about two and a half hours to complete.

4.3 Large Language Model

A common problem with data generation techniques is that nonsensical values are sometimes created; LLMs are nominally able to address this (5). We experiment with ChatGPT 4.1 Mini (7 billion parameters) to see if LLMs can generate usable data.

However, even from a highly engineered and iterated prompt. The initial test generations of 1,500 generated observations saw that data quality was extremely poor, with repeated rows and non-sensible values, for example, field labels instead of numerics. As such, we chose not to continue forward with the LLM data in downstream modeling.

4.4 Other Methods

We attempted to include a Tabular Variational Auto-Encoder, along with a Gaussian Copula, but because of technical limitations, software version incompatibility in the TVAE case and memory demands in the Copula case, we were not able to generate usable synthetic data from these methods.

5 Modeling with Synthetic Data

To test whether addition of synthetic data would improve our model we used 3 different methods of integrating synthetic data and tested the accuracy using our best model from our initial attempts: Light GBM. The 3 methods were the addition of the original ratio, the addition of ones, and synthetic only data. Each of these methods proved ineffective at consistently improving our model.

Do note that for all these analyses, the testing data was the original real test dataset and the resulting F1 and AUC scores are results from the models being tested on real data. To reiterate, the testing data contained no synthetic data at all.

5.1 Original Ratio

To test if adding synthetic data with the original ratio would improve the accuracy of our model we compared 4 different datasets each with a million entries. Base, which consists of 1 million entries randomly sampled from the original dataset, CTGAN Ratio, which contains 50% real data and 50% CTGAN generated data, DS Privacy, with 50% real data and 50% DS data with privacy

prioritized, and DS No Privacy with 50% real data and 50% DS data with no emphasis on privacy. Unfortunately, in our case, the base condition, training and testing only on real data, outperformed every other model, as shown in Figures 2 and 3 below.

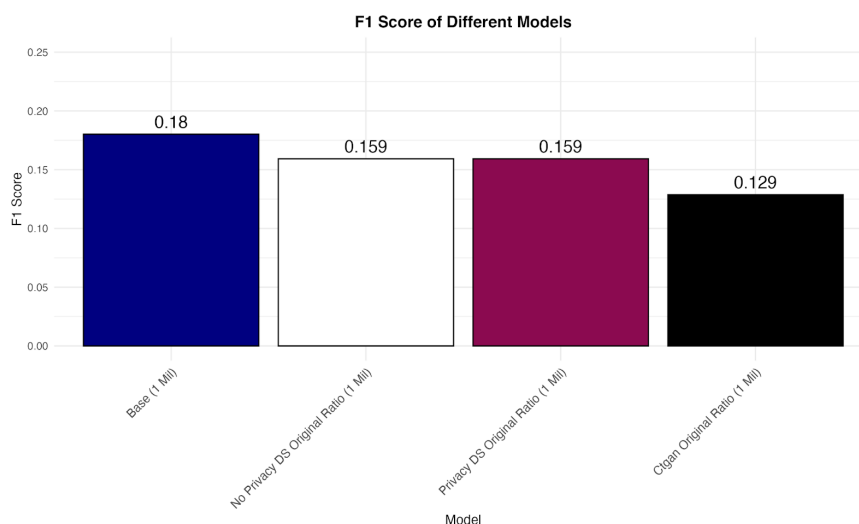


Figure 2. Base data outperforms synthetic-data augmented training data as measured by F1 Score.

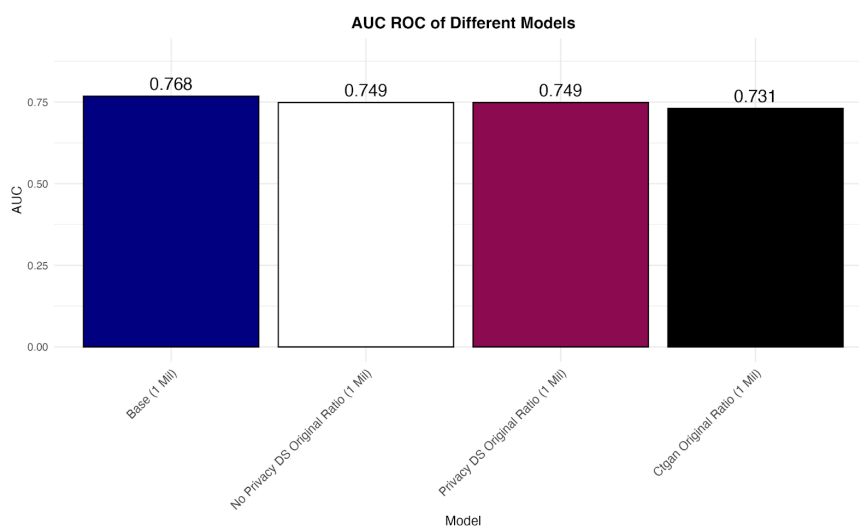


Figure 3. Base data outperforms synthetic-data augmented training data as measured by AU-ROC.

5.2 Synthetic Data Only

To test the robustness of the synthetic data on its own we looked at the performance of the synthetic datasets with the original ratio. The models looked at were Base with 550k entries randomly sampled from the original dataset, CTGAN Synthetic, which contains 500k CTGAN generated entries, DS Privacy, with 500k DS entries with privacy prioritized, and DS No Privacy with 500k DS entries with no emphasis on privacy.

As shown in the figures below, on their own, the synthetic datasets performed very poorly.

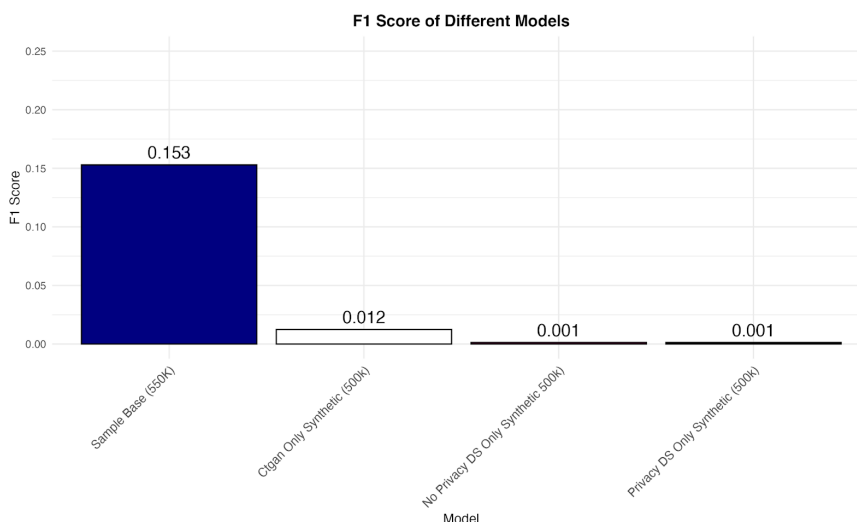


Figure 4. F1 scores for models trained exclusively on synthetic datasets compared to the real-data baseline.

This suggests that the fidelity of the synthetic data is limited, seeing as it has almost no predictive power on its own. However, it remains to be seen whether adding the synthetic positive cases alone might be able to improve model performance.

5.3 Adding Extra Positives

We assess the effectiveness of incorporating a higher ratio of synthetic ones in two ways.

First, to test if adding synthetic ones would improve the accuracy of our model we compared 4 different datasets each with 550k entries. Base, which consists of 550K entries randomly sampled from the original dataset, CTGAN Ratio, which contains 500k real entries and 50k CTGAN generated ones, DS Privacy, with 500k real entries and 50k DS ones with privacy prioritized, and DS No Privacy with 500k real data and 50k DS ones with no emphasis on privacy.

As shown in figures 6 and 7, DS with no privacy features enabled was on par with the base model in F1 Score and slightly outperformed it in AUC ROC. The rest of the models performed

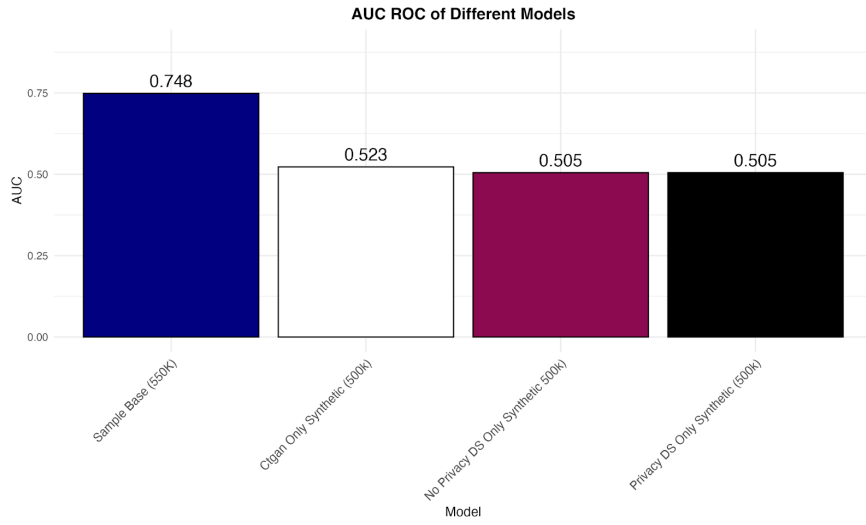


Figure 5. AU-ROC scores for models trained exclusively on synthetic datasets compared to the real-data baseline.

worse than the base model.

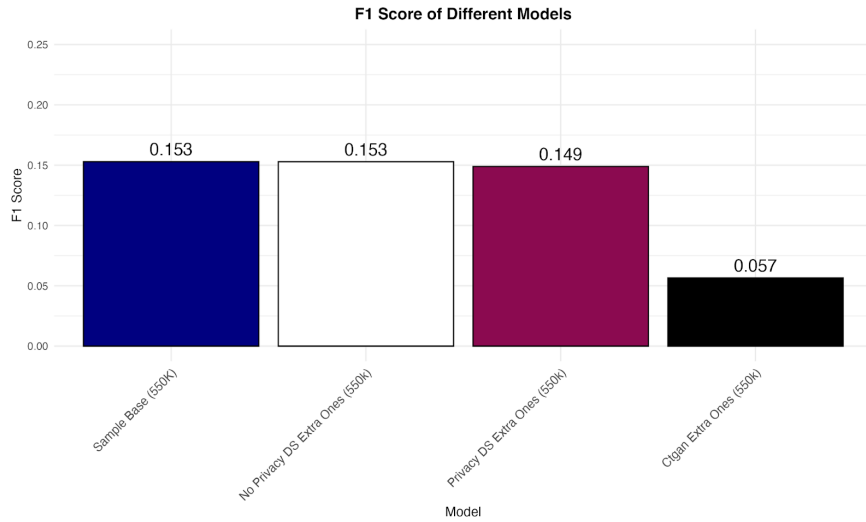


Figure 6. F1 scores when augmenting the training data with additional synthetic positive (label 1) cases.

5.4 Experiments with Synthetic Data

In addition to performing evaluation with the synthetic data, we ran two additional experiments that we outline below:

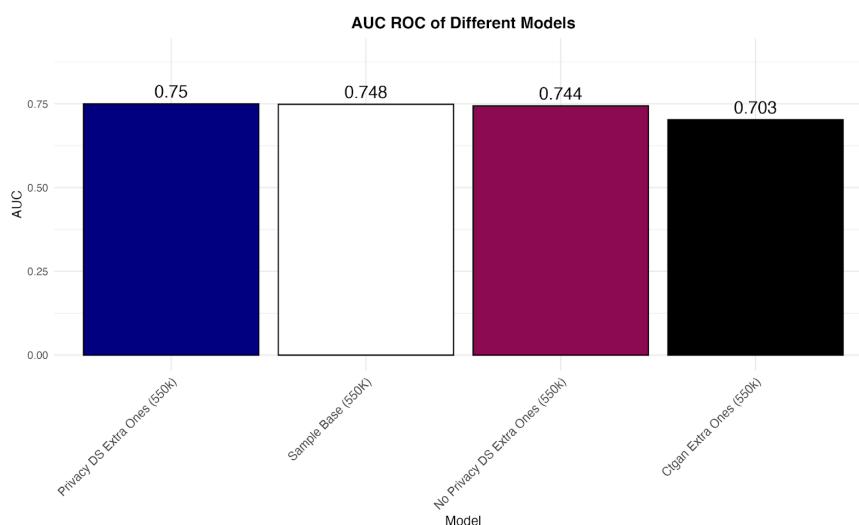


Figure 7. AU-ROC scores when augmenting the training data with additional synthetic positive (label 1) cases.

5.4.1 Experiment 1

In this experiment, we tested to see if gradually adding synthetically generated observations with the label 1 would improve F1 and AUC score. What we mean by this is that for each synthetic data type, we continuously added in 1s at different percentage levels of the training data size until the number of observations with label 1 matched the number of observations with label 0 (i.e. until the dataset became balanced). We wanted to test to see if as we approach balancing the label class with synthetic 1s (the minority class) if there would be a gradual improvement in F1 and AUC score. The results of our first experiment are shown below:

As we look at figure 8 and figure 9, we see the percentage of synthetic 1s added to the dataset on the x-axis and the relevant score on the y-axis. On the x-axis, 0% indicates the original distribution of 0s and 1s with no synthetic data added while 100% indicates that the number of 1s and 0s are perfectly matched with synthetic 1s and real 0s. 50% would indicate that 50% times the size of the training data is the number of synthetic 1s that were added into the dataset.

The results are not precisely as we expected as we see the AUC and F1 score do not change considerably as we attempt to balance the dataset with synthetic data. The scores also change very sporadically with no clear pattern. That said, it is possible to marginally improve the predictive power of the model vs. the baseline case, even if not consistently. In a world of fine margins, the difference between 0.073 and 0.078 may actually be meaningful across millions of ad impressions.

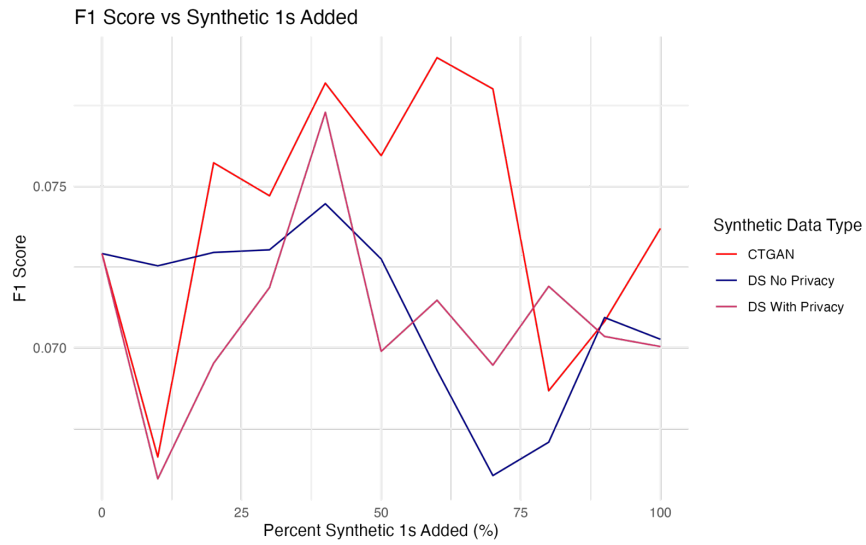


Figure 8. F1 score as a function of the percentage of synthetic positive cases added to the training data (Experiment 1).

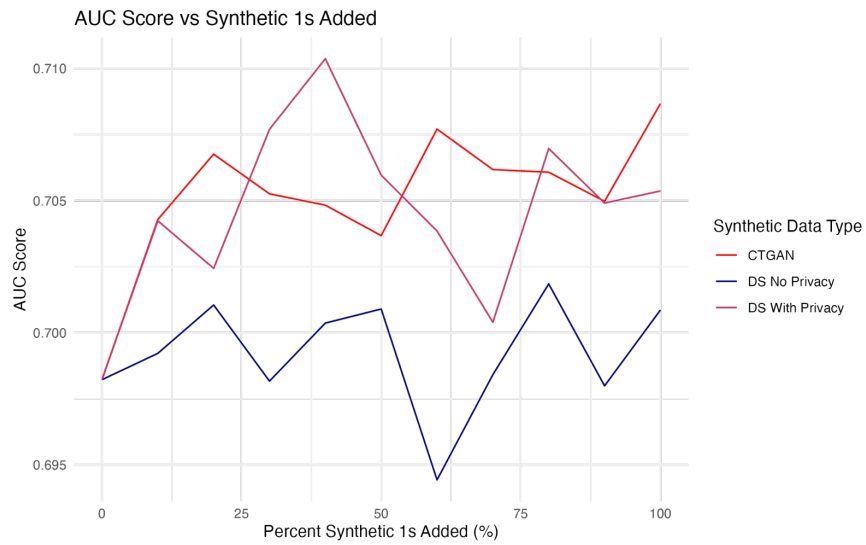


Figure 9. AU-ROC score as a function of the percentage of synthetic positive cases added to the training data (Experiment 1).

5.4.2 Experiment 2

In the second experiment, we tested whether mixing different ratios of synthetic data with real data would also lead to improved models. For this experiment, we took the fully synthetic data that contained synthetic 0s and 1s following the original data distribution and incrementally changed the

percentage of real and synthetic data that we trained on. We ensured that the original distribution of the label class was kept intact as we trained on different levels of synthetic and real data. The results of the experiment can be seen in figure 10 and figure 11

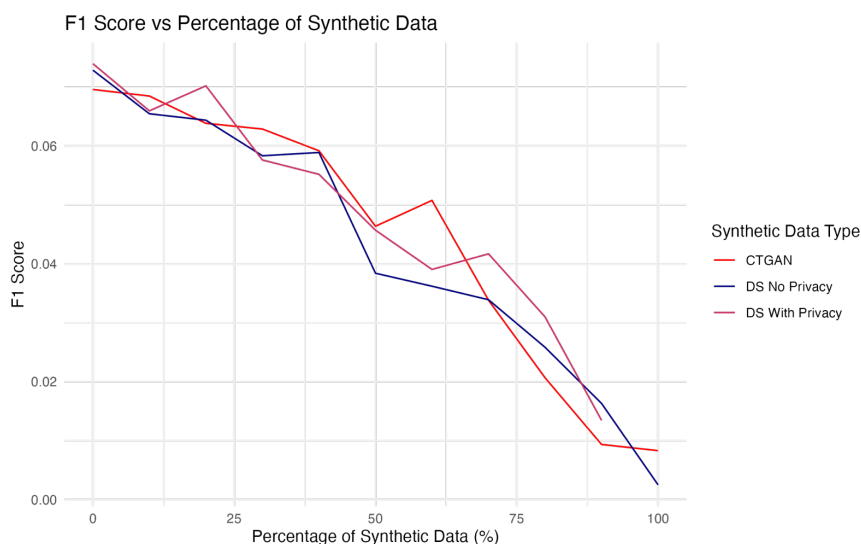


Figure 10. F1 score as the proportion of synthetic to real training data varies while preserving the original class distribution (Experiment 2).

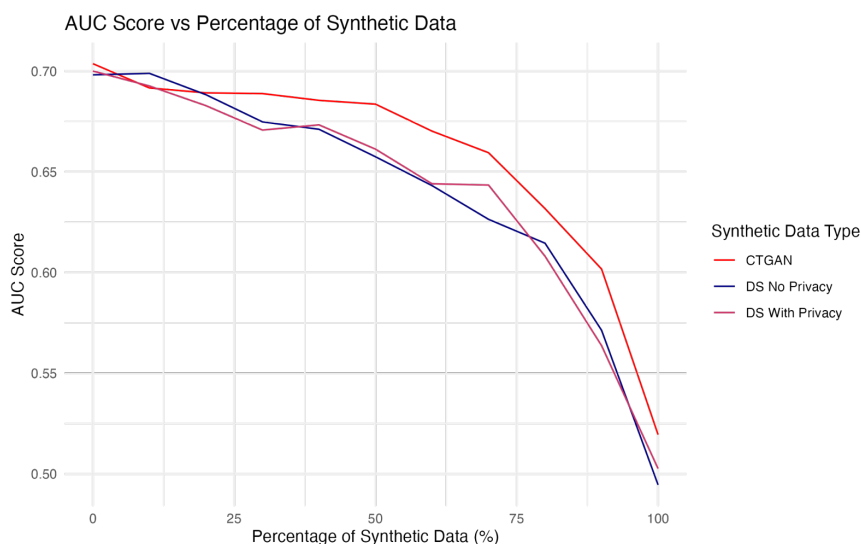


Figure 11. AU-ROC score as the proportion of synthetic to real training data varies while preserving the original class distribution (Experiment 2).

From the results, we can see a clear example of model collapse – as the percentage of data that

is trained on becomes fully synthetic, the F1 and AUC score degrade tremendously. 50% on the x-axis indicates that 50% of the data was synthetic and 50% was real while 75% indicates 75% of the data was synthetic while 25% of the data was real. We hoped to see that adding in synthetic data would help boost F1 and AUC scores but instead our results show that introducing synthetic data into the training data degrades the model.

5.5 Why Does Synthetic Augmentation Fail in This Setting?

Across all experiments, synthetic augmentation failed to meaningfully improve model performance. We identify several plausible mechanisms behind this outcome. We cannot determine with certainty which of these is the most important culprit. We nonetheless order these possibilities in rough order of how likely we believe them to be.

5.5.1 Sparse Signal in the Minority Class

With only 1.5% positive cases, the model sees extremely few examples of informative feature combinations associated with a click. Both CTGAN and DataSynthesizer must learn a conditional distribution that is itself poorly identified. The synthetic generators therefore reproduce the dominant (negative) structure of the data while adding substantial noise to the minority class. This makes the eventual training set, even when augmented with a substantial number of positive ones, still less able to reflect the signal that actually predicts ones in the testing set.

5.5.2 Feature–Label Entanglement is Weak

Inspection of preliminary models suggests that individual-level behavior, device information, and ad characteristics exhibit only weak association with click-through outcomes. When the true signal-to-noise ratio is low, synthetic models may simply amplify noise rather than enriching the signal.

5.5.3 Generators Overfit the Majority Regime

GAN-based models are known to collapse toward the mode of the distribution when faced with extreme imbalance. Our CTGAN results – particularly the poor performance of synthetic-only data – show evidence of mode collapse, with the generator producing unrealistic or non-informative 1’s.

5.5.4 Metric Mismatch Between GAN Objectives and Downstream Prediction

CTGAN optimizes a min–max adversarial objective, not a supervised one. Nothing in the training objective encourages the generator to create samples that are *useful* for a downstream

classifier. This disconnect is especially problematic for highly skewed binary classification.

5.5.5 Covariate Shift Introduced by Synthetic Data

Adding large volumes of synthetic data may change the empirical distribution of the training set. If this is indeed happening, the generators imperfectly capture conditional dependencies, and this alters joint feature distributions in ways that LightGBM does not generalize well to. As the proportion of synthetic data increases, the model effectively trains on a distribution that diverges from the test distribution, resulting in collapse.

6 Discussion

Our results highlight a fundamental challenge in applying synthetic data generation to extremely imbalanced behavioral datasets. Although synthetic data can be effective in clinical or financial contexts, where minority classes exhibit well-defined structure, click-through behavior in this dataset appears to contain weak and diffuse patterns. As a result, synthetic methods are unable to generate informative positive cases and may in fact dilute the already limited signal present in the real data.

An important implication is that class imbalance alone does not justify synthetic augmentation. The underlying data-generating process must contain a sufficiently strong and learnable signal for GAN- or Bayesian-based generators to recover. In the absence of such structure, synthetic samples behave primarily as noise injections, offering little benefit to downstream tasks.

Finally, our experiments suggest that future success may require additional sources of supervision (e.g., richer temporal features, user history embeddings, or ad-content representations) rather than simply more data. Synthetic approaches cannot compensate for structurally uninformative features.

7 Conclusion

We evaluate multiple generative approaches for producing synthetic positive observations in an extremely imbalanced advertisement click-through dataset. Despite using CTGAN, DataSynthesizer, and LLM-based generation, none of the synthetic augmentation strategies meaningfully improved model performance. In many cases, synthetic data degraded predictive accuracy, particularly when synthetic observations constituted a large share of the training set.

These results underscore a key insight: synthetic data generation is not a universal solution to class imbalance. When the minority class exhibits sparse structure or low signal-to-noise ratios, generators struggle to produce informative samples. Future work should explore alternative

approaches—such as representation learning, causal modeling of click mechanisms, or richer behavioral features—to address the underlying limitations rather than relying solely on synthetic augmentation.

References

- [1] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. Lightgbm: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/6449f44a102fde848669bdd9eb6b76fa-Paper.pdf.
- [2] Guang Cheng Din-Yin Hsieh, Chi-Hua Wang. Improve fidelity and utility of synthetic credit card transaction time series from a data-centric perspective. *arXiv preprint*, 2024. URL <https://arxiv.org/pdf/2401.00965>.
- [3] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. Smote: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16: 321–357, 2002. URL <https://arxiv.org/abs/1106.1813>.
- [4] Haoyue Ping, Julia Stoyanovich, and Bill Howe. Datasynthesizer: Privacy-preserving synthetic datasets. In *Proceedings of the 29th International Conference on Scientific and Statistical Database Management*, 2017. URL <https://faculty.washington.edu/billhowe/publications/pdfs/ping17datasynthesizer.pdf>.
- [5] Mengnan Du Xu Shen Yi Chang Ruxue Shi, Yili Wang and Xin Wang. A comprehensive survey of synthetic tabular data generation. *arXiv preprint*, 2025. URL <https://arxiv.org/pdf/2504.16506>.
- [6] Lei Xu, Ioannis Skandalis, Scharf Stein, et al. Modeling tabular data using conditional gan. In *Advances in Neural Information Processing Systems*, 2019. URL https://papers.nips.cc/paper_files/paper/2019/file/254ed7d2de3b23ab10936522dd547b78-Paper.pdf.
- [7] Xiaojiu1414. Digix global ai challenge dataset. Kaggle, 2023. URL <https://www.kaggle.com/datasets/xiaojiu1414/digix-global-ai-challenge>.
- [8] Guang Cheng. Classification slides, 2025. Course lecture slides.
- [9] Guang Cheng. Regression slides, 2025. Course lecture slides.

- [10] Guang Cheng. Week 8.1: Evaluation of synthetic data, 2025. Course lecture slides.
- [11] Yidong Ouyang. Lecture slides, 2025. Course lecture materials.
- [12] Google Brain Team. Tensorflow keras documentation. <https://www.tensorflow.org/guide/keras>, 2024.
- [13] Lei Xu, Ioannis Skandalis, Scharf Stein, and Kalyan Veeramachaneni. Modeling tabular data using conditional gan. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL <https://papers.nips.cc/paper/2019/hash/254ed7d2de3b23ab10936522dd547b78-Abstract.html>.
- [14] Haoyue Ping, Julia Stoyanovich, and Bill Howe. Datasynthesizer: Privacy-preserving synthetic datasets. In *Proceedings of the 29th International Conference on Scientific and Statistical Database Management (SSDBM)*. ACM, 2017. URL <https://faculty.washington.edu/billhowe/publications/pdfs/ping17datasynthesizer.pdf>.